

EDML—The Electrical Design Modeling Language

(c) 2022–2024 Altair Engineering Inc. All rights reserved.
concept-eevision@altair.com
Version 2025

Table of Contents

- 1. Introduction
- 2. A First Example
- 3. Language Reference
 - 3.1. Identifiers vs. Names
 - 3.2. Comments and Whitespace
 - 3.3. Shorthand Notation for Wires and Cavities
 - 3.3.1. Lists of IDs
 - 3.3.2. Generator expressions
 - 3.4. Attributes and Properties
 - 3.4.1. Attributes
 - 3.4.2. Properties
 - 3.4.3. Root Attributes and Properties
 - 3.5. Wires
 - 3.6. Components
 - 3.6.1. ECUs and Devices with Connectors
 - Connectors
 - Cavities
 - Component-internal electrical connections (Arcs)
 - 3.6.2. Connector-less Devices
 - 3.6.3. Inliners
 - 3.6.4. Splices and Eyelets
 - 3.6.5. User-defined Component Symbols
 - 3.6.6. Adding Labels to Components
 - 3.7. Multicores
 - 3.8. Modules
 - 3.8.1. Shorthand Notation
 - 3.9. Configurations
 - 3.10. Hierarchical Models
 - 3.10.1. Hierarchical Boxes (`HierBox`)
 - 3.10.2. Hierarchical ECUs (`HierComponent`)
 - 3.11. Internationalization
 - 3.12. Component Libraries
 - 3.12.1. Multiple Files per Project
 - 3.12.2. Component Definitions
 - 3.12.3. Parameterization
- 4. Tool Support
 - 4.1. The `edml2edb` Compiler
 - 4.1.1. Using the Compiler
 - 4.1.2. Licensing

4.2. The Visual Studio Code Extension

4.2.1. Installation

5. Extending and Modifying Existing Models

5.1. Describing Extensions

5.2. Calling the Compiler

5.3. Selecting Multiple Objects

5.4. Selection Conditions

5.5. Changeable Properties

5.6. Example

5.7. Order of Statements

5.8. Changing Component Types

Appendix A: EDML Keywords

A.1. Object Declaration

A.2. Properties and Fixed Property Values

1. Introduction

(
h
tt
p
s:
//
w
w
w
.
a
lt
a
ir.
c
o
m

The *electrical design modeling language* (or EDML for short) was developed by [Altair Engineering](http://www.altair.com)) to provide an easy-to-learn language for modeling electrical connectivity, e. g., of a vehicle, machine, or space craft. Using EDML engineers can easily create, debug, and document complex logical and physical wiring designs.

(
h
t
t
p
:
/
e
n
.
w
i
k
i
p
e
d
i
a
.
o
r
g
/
w
i
k
i/
H
a
r
d
w
a
r
e
-
d
e
s
c
r
i
p
t
i
o
n
_l
a
n
g

(
h
t
t
p
:
/
e
n
.
w
i
k
i
p
e
d
i
a
.
o
r
g
/
w
i
k
i/
V
e

(
h
t
t
p
:
/
e
n
.
w
i
k
i
p
e
d
i
a
.
o
r
g
/
w
i
k
i/

u	ri	V
a	l	H
g	o	D
e	g	L

In the semiconductor industry standardized hardware description languages (HDLs) like Verilog or VHDL have been widely used for decades to describe processor designs in a text-based format. EDML gives developers of electrical systems the same productivity gains and advantages that HDLs have given to semiconductor design engineers for many years.

Such modeling languages offer many advantages compared to, e. g., a graphical specification:

- As a text-based format, the description is *readable and modifiable by developers* without special tool support. Different powerful text editors are available — both as open source and commercial software — that allow writing EDML models conveniently.
- The textual description has *precisely defined semantics*, while, in a graphical tool, wires often cannot be distinguished from lines that serve only as a graphical decoration. This feature is the prerequisite for an automated analysis like sanity checks, for automatic rendering of schematics, and automatic document generation.

(
h
tt
p
s:
//
g
it
-
s
c
m
.c
o
m

- Keeping track of different *revisions of a design* is well supported by many different tools like [Git](#) /) or [Subversion](#)

(
h
tt
p
s:
//
s
u
b
v
e
r
si
o
n
.
a
p
a
c
h
e
.
o
r
g

/). The same holds for comparing different revisions and highlighting the changes.

- EDML offers the possibility to define component libraries. Such libraries allow the reuse of common components and can easily be shared between different projects.

An EDML file is a UTF-8 encoded text file describing the devices and electrical connections in the system. Typically the file extension “.edml” is used for EDML files. As an optional component, EEvision provides a compiler (`edm12edb`) that converts EDML files to EEvision’s database format EDB, which can be loaded and visualized in EEvision.

To debug and explore your design with EEvision, please make sure you copy the created EDB file and, if used, the referenced SVG, PNG, and JPG image files into the `standalone/` directory or a into a project directory `standalone/<project>/` in your EEvision installation path.

2. A First Example

Before we go into the details of EDML, we introduce the main concepts using a simple example.

Assume we want to model the system shown in Figure 1 using EDML.

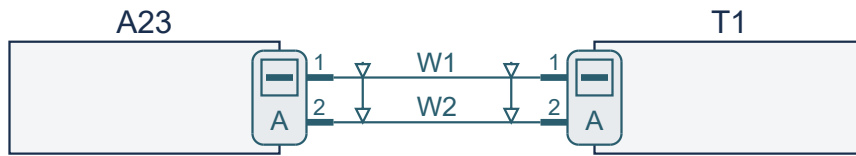


Figure 1. A first example

The example consists of two components with names `A23` and `T1` as well as two twisted wires `w1` and `w2`. The corresponding EDML model that describes this system looks as follows:

Listing 1. Creating two components that are connected by two twisted wires

```
Wire W1;
Wire W2;

Component T1;
  Connector A;
    Cavity 1;
    Cavity 2;
  Join A.1 -> W1;
  Join A.2 -> W2;

Component A23;
  Connector A;
    Cavity 1, 2;
  Join A.1 -> W1, A.2 -> W2;

Multicore MC1 (W1, W2) | Type = twisted;
```

EDML

The first line `Wire W1;` creates a single wire named `w1`. `w1` is both displayed in the schematics as the name of the wire and used as an identifier (ID) for referring to this wire within the EDML file, e. g., when connecting it to a component or making it part of a wire bundle (also called a “multicore”). Every statement ends with a semicolon. The second wire `w2` is created in the same way.

NOTE

Identifiers must consist of digits, lowercase and uppercase letters, and underscores only. We will see later how we can use arbitrary strings that are not valid IDs as the displayed name of the object.

Next, we create the two components. ECUs and other devices (like the ones shown in Figure 1) are created using the keyword `Component`. Other keywords exist for other types of components like splices. That means, the line `Component T1;` creates a new ECU component with the name `T1`. Additionally the identifier `T1` can be used to

refer to this component.

Connectors are added to a component using the `Connector` keyword, followed by an identifier for the connector; in this case, `A`.

We add cavities to the connector `A` using the keyword `Cavity` followed by an identifier for the cavity. We repeat this statement for the second cavity.

Finally, we connect the two wires that we have already defined to the cavities of the component using the `Join` keyword. `Join A.1 → w1`; means that the wire with identifier `w1` is connected to the cavity with identifier `1` of the connector `A` of the current component. The second wire `w2` is connected to cavity `2` of connector `A` in a similar way.

NOTE

Every object must be declared before its ID can be used to refer to this object. For instance, the wires that are connected to a component must be created using the `Wire` statement before the component, because the `Join` statement, which is part of the component declaration, needs to refer to these wires.

The creation of the second component named `A23` happens analogously. For the definition of the cavities we use a shorthand notation: `Cavity 1, 2`; is equivalent to

```
Cavity 1;  
Cavity 2;
```

EDML

The same applies to the `Join` keyword.

The last statement that needs discussion here is the definition of the twisted wire bundle ("multicore"). Multicores are always defined using the `Multicore` statement followed by an identifier for the multicore (here: `MC1`). The part `(w1, w2)` tells us that the multicore encompasses the wires with identifiers `w1` and `w2`. EDML supports different kinds of multicores: Wires can be twisted, shielded, or both. Therefore we need to specify which type of multicore we want to have, in this case a twisted multicore. The list of properties and attributes of an object is always separated by a vertical bar "|" from the object ID. The keyword `Type` is used to specify the type of the multicore, in this case its type is `twisted`.

If we store the EDML code in a file named `example.edml`, we can use the `edm12edb` compiler to generate an EDB database and use EEvision to debug and visualize the design.

```
# Linux:  
edm12edb example.edml -o example.edb  
  
# Windows:  
edm12edb.exe example.edml -o example.edb
```

EDML offers the possibility to create component libraries and store them in separate files. The components in such a library can not only be reused in several projects, but also be instantiated multiple times in the same dataset. For instance, if a vehicle contains several identical relays, it suffices to define the relay once and reuse it whenever as often as needed.

In our example, we could move the components to their own file:

Listing 2. An EDML component library

```
Define {  
    Component Sensor;  
        Connector A;  
            Cavity 1;  
            Cavity 2;  
}  
  
Define {  
    Component Controller;  
        Connector A;  
            Cavity 1,2;  
}
```

EDML

Using the `#include` directive, we can make the component definitions available in another file. We can then compactly write our example as follows:

Listing 3. Using components from an EDML library file

```
#include "library.edml"  
  
Wire W1, W2;  
  
Sensor T1;  
    Join A.1 -> W1;  
    Join A.2 -> W2;  
  
Controller A23;  
    Join A.1 -> W1, A.2 -> W2;  
  
Multicore MC1 (W1, W2) | Type = twisted;
```

EDML

3. Language Reference

EDML provides a set of basic elements for modeling electrical connectivity: wires, different kinds of components, multicores for modeling wire bundles like twisted or shielded sets of wires, and modules for grouping sets of objects.

3.1. Identifiers vs. Names

As we have seen in our previous example in Listing 1 every object requires an identifier (ID) that can be used to refer to this object.

An ID is a non-empty sequence of digits (0–9), lower- and uppercase letters (a–z, A–Z), and underscores (_). Note that IDs are case-sensitive. The IDs of all wires, components, multicores, and modules (which have not been used in Listing 1) have to be unique. IDs of connectors have to be unique within their component. For components with connectors, the IDs of cavities are required to be unique within their connector; for components without connectors (like splices), the IDs of cavities have to be unique within the component.

Every ID needs to be defined before it can be used to refer to its object. For instance, a wire needs to be defined before the component to which it is connected because the `Join <cavity ID> → <wire ID>;` statement uses the ID to refer to the wire.

By default, if no other name is specified, the ID is also used as the name of the object and is displayed in EEvision. If that is not possible (for instance, because the name contains spaces or is not unique) or desired, a different name for the object can be specified as a property as follows:

```
Component S1 | Name="Temperature Sensor";
```

EDML

For the case when an object should not get a name at all, there is a special notation that omits the value of the `Name` property:

```
Wire W1 | Name=;
```

EDML

This line creates a wire with ID `w1`, but without any name. Technically, not creating any name is different from using an empty string as the name:

```
Wire W1 | Name="";
```

EDML

Both are valid statements, but from a practical perspective, there is little difference between the two.

3.2. Comments and Whitespace

The syntax of EDML allows to add *comments* to a file. They are not evaluated by the compiler. Similar to programming languages like C, C++, and Java, EDML allows two styles of comments:

- Single-line comments start with `//` and extend until the end of the current line.
- Multi-line comments start with `/*` and end with the next `*/`. Multi-line comments cannot be nested.

Listing 4 shows an EDML model in which comments are used to give general information about the source file and to document the purpose of different objects in the model.

Listing 4. Using comments to document the EDML model

```

/*****
 * Harness model of our new vehicle
 * Copyright 2021 by SuperCar Inc.
 * Author: Jane Doe
 *****/

// Declare the power network
Wire Wp(1:100) | Type = power; //TODO: add 'length' information.

// Declare the ground network
Wire Wg(1:78) | Type = ground;

// The central ground splice of the car
Splice S_gnd | "Position" = "center";
    Cavity 1,2,3,4,5;

// ...

```

EDML

Whitespace outside of string constants (the part between double quotes) is mostly ignored—the compiler merges sequences of spaces, tabulators, and newlines into a single space. For instance, all of the following statements are equivalent:

```

Wire W1 | Name="Temperature Sensor";
Wire W1 | Name = "Temperature Sensor";
Wire W1
    | Name = "Temperature Sensor";

```

EDML

3.3. Shorthand Notation for Wires and Cavities

Since wires and cavities are the most frequently defined objects in an EDML file, EDML allows shorthand notations for defining wire and cavities and connecting them to each other.

3.3.1. Lists of IDs

Both the `wire` and the `cavity` statement accept a list of IDs instead of only a single ID like other object declarations. For instance

```

Wire Wground;
Wire Wpower;
Wire Wdata;

```

EDML

can be written compactly as follows:

```
Wire Wground, Wpower, Wdata;
```

EDML

All attributes and properties in such a definition refer to *all* IDs in the statement, i. e.,

```
Wire Wground, Wpower, Wdata | Color="red", "Diameter"="2 mm";
```

EDML

creates three wires that have a red color marker and an attribute `Diameter` with value `2 mm`.

3.3.2. Generator expressions

In particular the cavities of a connector are often numbered sequentially. *Generator expressions* provide a convenient way to create wires and cavities. Their syntax is

```
<Prefix>(<n>:<m>)<SUFFIX>
```

TEXT

where `<PREFIX>` and `<SUFFIX>` are arbitrary (possibly empty) strings consisting of digits, letters, and underscores; `<n>` and `<m>` are integer numbers with $0 \leq n \leq m$.

This expression is equivalent to writing

```
<PREFIX><n><SUFFIX>, <PREFIX><n+1><SUFFIX>, ..., <PREFIX><m><SUFFIX>
```

TEXT

For instance, `W(1:5)_out` yields `W1_out`, `W2_out`, `W3_out`, `W4_out`, `W5_out`. The following listing shows how these generator expressions are used.

Listing 5. Generator expressions for wires and cavities

```
Wire W(1:10)_out;
```

EDML

```
Component S1;
```

```
    Connector A;
```

```
        Cavity (1:10);
```

```
    Join A.(1:10) -> W(1:10)_out;
```

```
Component A23;
```

```
    Connector A;
```

```
        Cavity x(1:10);
```

```
    Join A.x(6:10) -> W(1:5)_out, A.x(1:5) -> W(6:10)_out;
```

```
Multicore MC1 (W(5:8)_out) | Type = shielded;
```

The first line generates ten wires `W1_out`, `W2_out`, ..., `W10_out`. Similarly, Line 5 yields ten cavities at the connector `A` of component `S1`; their IDs (and names) are `1`, `2`, ..., `10`. The second `Cavity` statement in Line 10 creates cavities with IDs and names `x1`, `x2`, ..., `x10`.

Line 6 (`Join (1:10) → w(1:10)_out;`) connects all ten cavities of `s1` to the ten wires. The first element of the left generator expression is connected to the first element of the right one, the second to the second etc. Therefore the generator expressions left and right of the arrow need to have the same width; otherwise the compiler reports an error. As we can see in Line 11, the ranges of the two generator expressions do not have to be the same, only their widths have to match.

The resulting schematic is shown in Figure 2.

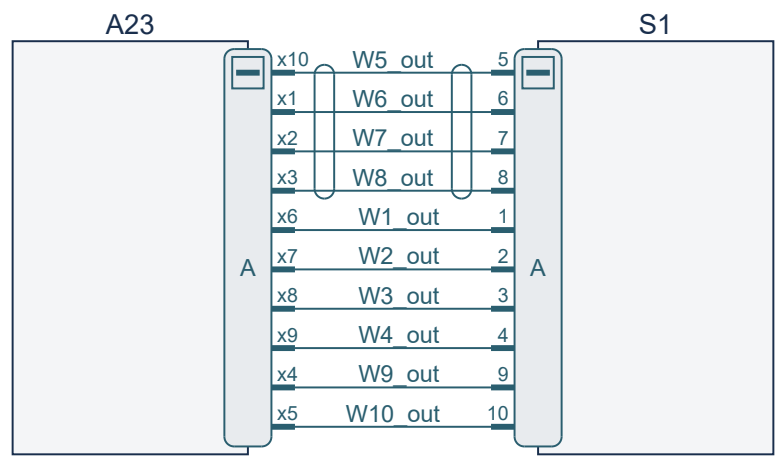


Figure 2. Schematic obtained from Listing 5 using generator expressions

3.4. Attributes and Properties

Object definitions can be extended with additional information using attributes and properties.

3.4.1. Attributes

Every object (wire, component, connector, cavity, multicore, module) in an EDML file can be enriched with an arbitrary number of user-defined *attributes*. An attribute is a name-value pair, where both name and value can be arbitrary strings, enclosed in double quotes and separated by an equality sign. Note that attributes whose name starts with a space character are reserved for internal purposes. Both name and value of an attribute may consist of several lines of text. The lines need to be separated using `\n`. If the name or value contains a double quote, it must be specified as `\` . Several attributes with the same name are allowed.

One can access the additional information in EEvision by selecting the object and then opening the info window or by opening the balloon window with a right-click on the object (the latter requires the “balloon” plugin to be activated).

To see the effect of attributes, let us replace in Listing 1 the first line `wire w1;` with the following code:

Listing 6. Wire enriched with attributes

```
Wire w1 | "Diameter" = "2mm",
        "Length" = "20cm", "Length" = "7.87in",
        "Material" = "Copper\n\"Gold\"";
```

Then EEvision displays the schematics as shown in Figure 3 (here with the attributes listed in the balloon window).

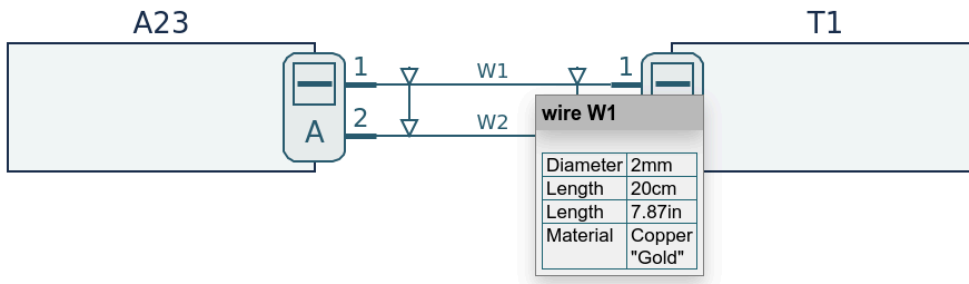


Figure 3. Wire enriched with attributes

3.4.2. Properties

In contrast to attributes, *properties* influence how objects are displayed in EEvision. Besides the property `Name`, we have already seen the property `Type` in Listing 1:

```
Multicore MC1 (W1, W2) | Type = twisted;
```

EDML

It tells the compiler that we want to see a twisted pair of wires. If we replace this line by

```
Multicore MC1 (W1, W2) | Type = shielded;
```

EDML

the two wires are displayed as shielded instead (cf. Figure 4).

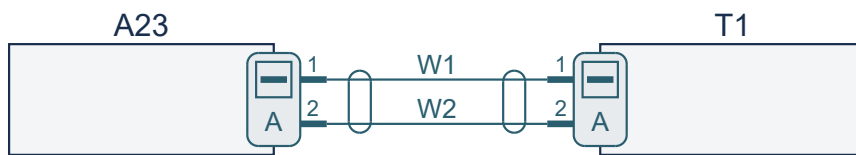


Figure 4. A shielded pair of wires

Properties look similar to attributes, but they have fixed names that are not enclosed in double quotes. The value of a property is either pre-defined and not enclosed in quotes (like in `Type=twisted`) or a user-defined string, written in double quotes (like `Name="Temperature Sensor"`). Sometimes the user-defined string needs to follow a certain syntax, for instance, the `Color` property of components requires that the value is a color description. So it must be either one of the pre-defined color names like `red` or `paletvioletred` or a hexadecimal RGB-value like `#FF0000` (the same as `red`) plus an optional transparency specifier like in `#FF0000/7F` or `red/50%` for half-transparent red.

Which properties are supported depends on the type of the object (with the exceptions listed below in Table 1). Also the allowed values are often specific for the object type. We provide a table with the supported properties, their allowed values, and their meaning when we describe the different objects.

The following table lists those properties that are supported by all object types. The first column contains the name of the property, the second column the allowed values. An entry `"string"` means that an arbitrary string is supported; `"syntax"` stands for a user-defined string that needs to have a certain format. Otherwise the supported fixed values are listed. The column `"Repeat"` specifies whether the property is allowed to occur multiple times at the same object or not.

Table 1. Properties supported by all EDML objects

Property	Values	Meaning	Repeat
Href	"syntax"	Allows to display a link to an URL in the info or balloon window.	yes
Image	"syntax"	Allows to display an image in the info or balloon window.	yes
Name	"string" or nothing	Specifies the displayed object name.	no
Subtype	"string"	Specifies a sub-type for the object	no
Video	"syntax"	Allows to display a video clip in the info or balloon window.	yes

The `Name` property has already been introduced in Section 3.1. The three properties `Href`, `Image`, and `Video` are used to generate special kinds of attributes, which are shown in EEvision in the info window and the balloon window. Their syntax is as follows:

```
Wire W1 |
  Href = "<name>,<url>,<text>",
  Image = "<name>,<filename>,<width>,<height>",
  Video = "<name>,<filename>,<width>,<height>";
```

EDML

- The `Href` property creates a clickable link in the info and balloon window of EEvision. It expects a comma-separated list with three entries: the first one is the name of the special attribute, the second a URL, and the third the text that is shown as the attribute's value.
- The `Image` property is used to display an image in the info and balloon window. It expects a comma-separated list with four entries: `<name>` is the text that should appear in the attribute name column; `<filename>` is the name of the image file. It must be relative to the directory that contains the EDB file. Finally, `<width>` and `<height>` specify the size of the image. Both are dimension-less numbers and measured in pixels. Alternatively the value `auto` can be used for either width or height or both. In this case, the image is scaled automatically to the given dimension or its natural size, preserving its aspect ratio.
- The `video` property is similar to the `Image` property, but shows a video in the info and balloon window instead. The syntax is the same as for the `Image` property.

Consider the following example:

Listing 7. A component with a link, an image, and a video

```
Component S1 | Name = "Temperature Controller",
  Href = "Vendor,https://www.concept.de,Concept Engineering GmbH",
  Image = "Image of the ECU,ecu.png,200,auto",
  Video = "Instructions,instructions.mp4,250,150";
Connector A;
Cavity 1,2,3;
```

EDML

Figure 5 shows the resulting entries in the balloon window of EEvision.

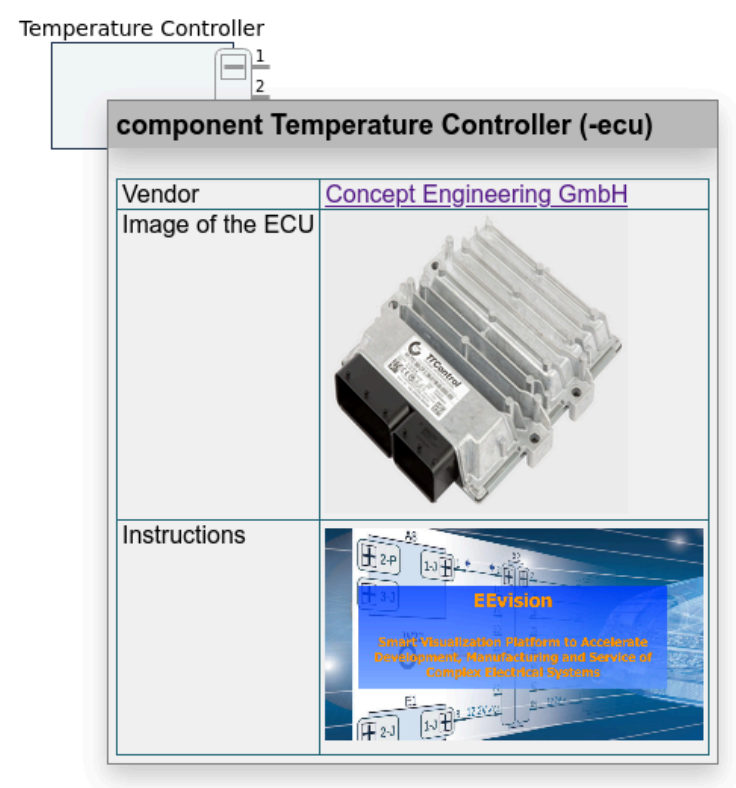


Figure 5. Component with a link, an image, and a video

Finally, the property `Subtype` allows to assign to each object a sub-type, which is used by EEvision to group objects of the same type (Wire, Multicore, etc). The sub-type does not have any influence on how objects are rendered.

3.4.3. Root Attributes and Properties

Not only the objects in the database can be equipped with attributes and properties, but also the database itself. Adding database attributes is done using the `Attributes` statement.

Listing 8. Creating root attributes and properties

```
Attributes Index = "Author\tRevision",
    "Author" = "John Doe",
    "Revision" = "1.1-beta";

Wire W1;

// ...
```

EDML

This example creates two root attributes `Author` with value `John Doe` and `Revision` with value `1.1-beta`. Additionally, it uses the property `Index` to tell EEvision (or rather its file select page) to list the values of the attributes `Author` and `Revision` when displaying the dataset details is enabled. To do so, the attribute names are listed as the value of the `Index` property, separated by tabulators. Note that tabulators are specified in EDML as `\t`.

List of Available EDB Files
☒ show details

Filename	EEvision	Author	Revision
ex1.edb	start	filter	
ex2.edb	start	filter	
ex3.edb	start	filter	
ex4.edb	start	filter	
ex5.edb	start	filter	John Doe 1.1-beta
ex6.edb	start	filter	

Alternatively [upload local file](#) (see also: [KBL files](#)).

We make things visible®
Copyright © 2018–2021 by [Concept Engineering](#). Last Modified: 2021/09/03.

Figure 6. EEvision's file select page showing the "Index" attributes

Besides the properties listed in Table 1, the following root property is supported:

Table 2. Supported root properties

Property	Values	Meaning	Repeat
Index	"syntax"	Tab-separated list of attribute names that can be displayed on the file selection page.	no

3.5. Wires

As we have already seen in different examples, wires are created using the `wire` statement, followed by a single ID, a list of IDs, or a generator expression.

```
Wire W17 | Name = "-W17", Type = ground,
        Color = "red seagreen darkorchid", "Signal" = "Ground";
```

EDML

Connecting wires with components is done as part of the component definition (see Section 3.6).

Table 3. Supported properties of wires

Property	Values	Meaning	Repeat
Color	"syntax"	Displays information about the wire's cover color as a small box on the wire.	no
Style	"string"	associates a wire with a certain style	no

Property	Values	Meaning	Repeat
Type	power , ground , logical , bus , hv	Defines the type of the wire.	no

(
h
tt
p
:/
/
w
w
w
.
w
3
.
o
r
g
/
T
R
/
S
V
G
1
1
/t
y
p
e
s.
h
t
m
l

C
o
l
o
r
K
e
y
w
o
r
d
s

The value of the `color` property is a space-separated list of color names like `red` or `darkorchid` or

hexadecimal RGB color values of the form #RRGGBB .

For semi-transparent colors, the hexadecimal format can be appended by a slash and two more hex digits, defining the transparency in a range from 00 (= opaque) to FF (= fully transparent). For example: #FFA500/7F defines a 50% transparent orange color. The named colors above can be appended by a slash and a decimal number in the range from 0 (= opaque) to 100 (= fully transparent) and a percent sign. For example: orange/50% defines a 50% transparent orange color. In the context of wire markers, transparency is rarely used.

For wires, up to three colors can be specified. Listing 9 defines three wires with different color markers. The corresponding schematics can be seen in Figure 7.

The property `Type` can be used to categorize the wires into power (`Type=power`) and ground wires (`Type=ground`), buses (`Type=bus`), high-voltage wires (`Type=hv`), and logical wires (`Type=logical`). While unclassified wires are always drawn in EEvision with black lines, the user can choose arbitrary colors for the different wire types.

Additionally, the classification of wires as power and ground wires influences EEvision’s extraction algorithms. For instance, the user can tell EEvision to ignore all power wires when extracting the electrical connectivity of a component.

In Listing 9, the wire `w2` is categorized as a power wire, and wire `w3` as a ground wire. In EEvision, we have assigned the color red to power and brown to ground wires (see Figure 7).

Listing 9. Wires with color markers and type classification

```
Wire W1 | Color = "red seagreen darkorchid";  
Wire W2 | Type = power, Color = "#FFA500 black";  
Wire W3 | Type = ground, Color = "red/50% #F0B75A/7F";
```

EDML

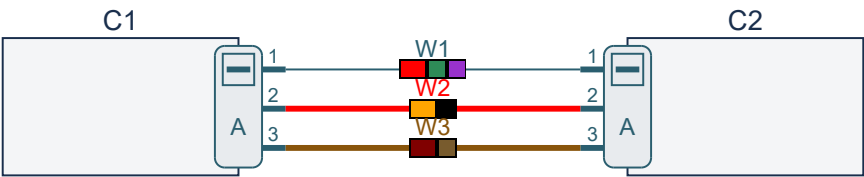


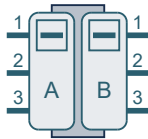
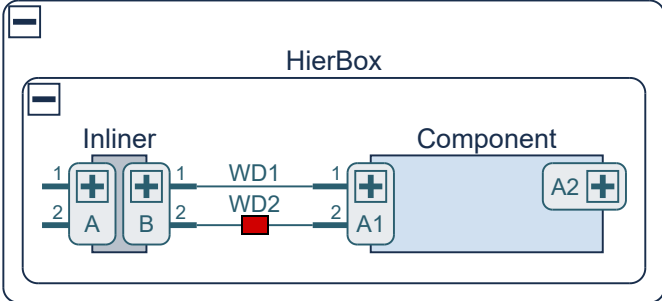
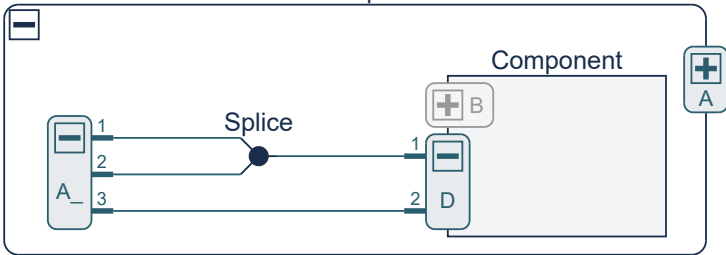
Figure 7. Wires with color markers and type classification

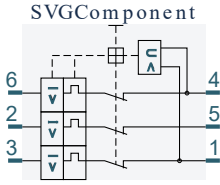
3.6. Components

EEvision supports different kinds of components in an electrical network; the commonly used ones are described in detail here; hierarchical components follow in Section 3.10. Table 4 gives an overview of all available component types and how they are displayed in EEvision. The first column contains the EDML keyword that is used to create a certain type of component, the second column the typical visual representation in EEvision, and the last column a link to the section where the details of this type of component can be found.

Table 4. Component types supported by EDML/EEvision

Keyword	Shape	Described in
---------	-------	--------------

Keyword	Shape	Described in
Component	Component 	Section 3.6.1
LComponent	LComponent 	Section 3.6.2
Inliner	Inliner 	Section 3.6.3
Splice	Splice 	Section 3.6.4
Eyelet	Eyelet 	Section 3.6.4
HierBox	HierBox 	Section 3.10.1
HierComponent	HierComponent 	Section 3.10.2

Keyword	Shape	Described in
SVGComponent		Section 3.6.5

3.6.1. ECUs and Devices with Connectors

The basic device is created using the keyword `Component` . It is typically used for ECUs, sensors, motors, and many more. The main characteristic is that wires are connected to such a device via connectors. Examples of such components are contained, e. g., in Listing 1 and Listing 5.

Besides the general properties in Table 1, `Component` objects support additional properties, which are listed in Table 5.

Table 5. Supported properties of `Component`s` and `LComponent`s`

Property	Values	Meaning	Repeat
Color	"syntax"	Displays the component with the specified color.	no
Imagedsp	"syntax"	Can be used to display an image file in the component’s body.	no
Style	"string"	Associates a component with a certain style	no

The `Color` property is used to assign a certain color to the component. It follows the same syntax as the `Color` property of wires (see the syntax description following Table 3) with the exception that one or two color values are accepted. In case of a single value, it specifies the component’s fill color; if two values are given, the first is the fill color, the second the border color. If only the border color should be set, the first color may be specified as “?”.

The property `Imagedsp` (do not confuse it with `Image` !) can be used to display an SVG, PNG, or JPG image inside the components body. The format of the string is one of the following three possibilities:

```

"<filename>,<width>,<height>".
"<filename>,<width>"
"<filename>"

```

TEXT

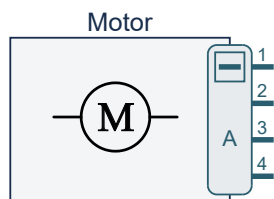
where `<filename>` is the name of the image file; `<width>` and `<height>` are dimension-less numbers specifying width and height of the image in pixels. If `<height>` or both are missing, 100 is used as the default.

For instance, a component with a small “Motor” icon could look as follows:

```
Component Motor | Imagedsp = "motor.svg,75,75";
```

```
Connector A;
```

```
Cavity (1:4);
```



EEvision is shipped with a few built-in symbols, loosely following the German standard DIN 40719-2. All of them have a one letter name, which is specified in the `Imagedsp` property *without file extension*. That means `Imagedsp="A,50,50"` uses the built-in symbol `A`, while `Imagedsp="A.svg,50,50"` uses the user-provided file `A.svg` for the symbol.

Table 6 shows the list of available symbols.

Table 6. EEvision's built-in symbols

Name	Image	Typical use
A		Assembly groups, boards, ECUs
C		Capacitors
E		Heating, lights
F		Protective devices like fuses
G		Generators and power supplies
H		Annunciators, both optical and acoustic
K		Relays
L		Inductors
M		Motors

Name	Image	Typical use
P		Measurement devices, counters
R		Resistors
S		Switches
V		Tubes or semiconductors like diodes, transistors, rectifiers
W		Transmitters like antennas, cables, optical fibers, wave guides etc.

Connectors

`Component` objects require connectors (like `Inliner` and `HierComponent` objects), before cavities can be defined that are used as connection points for wires. They are created using the keyword `Connector` following the component definition. A component can have an arbitrary number of connectors (see Listing 10 for an example).

Table 7. Supported properties of connectors

Property	Values	Meaning	Repeat
Color	"syntax"	Displays the connector with the specified color.	no
Style	"string"	Associates a connector with a certain style	no
Type	male , female , invisible , anti	The value <code>anti</code> is only available for <code>Inliner</code> and <code>HierComponent</code> objects.	no

The `Color` property is completely analogous to the corresponding property at the component.

The `Type` property specifies whether the connector is a female connector (jack) or a male connector (plug). A connector can also be specified as invisible, e. g., if a cable is directly attached to the component without a connector in between (“pigtail components”). The value `anti` is reserved for connectors of inliner and hierarchical components. For details, see Section 3.6.3 on inliners.

How the different types of connectors are displayed in EEvision can be seen in Figure 8: **A** is a connector with unspecified type; it is displayed with rounded corners. **B** is a male connector, displayed with a skewed edge. Female connectors like **C** have four rectangular corners. Finally, connector **D** is invisible and not shown in the schematic; such invisible connectors still keep the cavities together in a block.

Listing 10. A component with connectors of different types

EDML

```
Component ECU;
  Connector A;
    Cavity (1:3);
  Connector B | Type = male;
    Cavity (1:3);
  Connector C | Type = female;
    Cavity (1:3);
  Connector D | Type = invisible;
    Cavity (1:3);
```

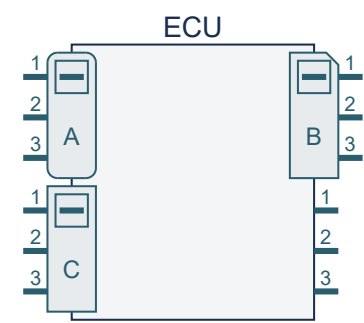


Figure 8. A component with connectors of different types

Cavities

Cavities are the connection points for the wires at the component. For `Component` objects they belong to the connectors, not to the component itself. The following properties are supported for cavities:

Table 8. Supported properties of cavities

Property	Values	Meaning	Repeat
Type	in , out , halfdot , spliced	Direction and shape of a cavity.	no
Ecfile	"syntax"	Used to control the display of the inner circuitry of a component. Only supported at cavities of <code>Component</code> and <code>LComponent</code> objects.	no

Listing 11. Cavities of different types

```
Wire W(1:5);
```

```
Component C1;
```

```
Connector A;
```

```
Cavity 1;
```

```
Cavity 2 | Type = in;
```

```
Cavity 3 | Type = out;
```

```
Cavity 4 | Type = halfdot;
```

```
Cavity 5 | Type = spliced;
```

```
Join A.(1:5) -> W(1:5);
```

```
Component C2;
```

```
Connector B;
```

```
Cavity 1;
```

```
Cavity 2 | Type = out;
```

```
Cavity 3 | Type = in;
```

```
Cavity 4 | Type = halfdot;
```

```
Cavity 5 | Type = spliced;
```

```
Join B.(1:5) -> W(1:5);
```

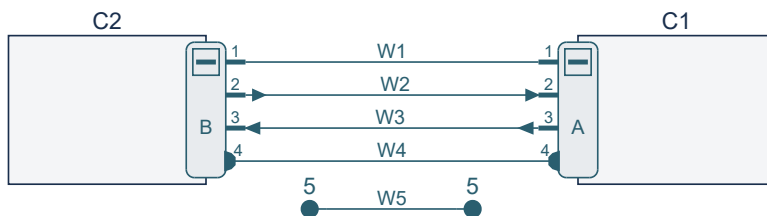


Figure 9. Cavities of different types

The types `in` and `out` specify the signal direction, shown as small arrows on the wire connected to the cavity. The `halfdot` type yields cavities with a semi-circle shape. They are typically used for shield connections. Finally, the `spliced` type creates a splice-shaped cavity that is detached from the component. They are needed for technical reasons to overcome some limitations of multicores and only mentioned here for the sake of completeness.

The property `Ecfile` is used by EEvision's `endcircuit` plugin to display the inner circuitry of a component. It is only supported at cavities of `Component` and `LComponent` objects. For details how to use it, please consult the documentation of the `endcircuit` plugin.

Component-internal electrical connections (Arcs)

Arcs are internal electrical connections between the cavities of a component. For instance, in a fuse with two cavities, the two cavities are electrically connected (at least as long as the fuse is not blown). Such internal connections can be modeled in EDML using the `Arc` keyword. An arc can encompass an arbitrary number of cavities of the same component.

EEvision does not display the arcs in the schematics, but its extraction algorithms take them into account, e. g., when determining the electrically connected wires.

Listing 12. A component with arcs between two pairs of cavities

```

Component Switch;
  Connector A;
    Cavity IN | Type = in;
    Cavity CTL | Type = in;
  Connector B;
    Cavity OUT1 | Type = out;
    Cavity OUT2 | Type = out;
  Arc A1 (A.IN, B.OUT1);
  Arc A2 (A.IN, B.OUT2);

```

Figure 10 shows three components where the component `Switch` is modeled as shown in Listing 12 with two arcs between cavities `IN` and `OUT1` as well as `IN` and `OUT2`. The wires that EEvision recognizes as electrically connected are highlighted.

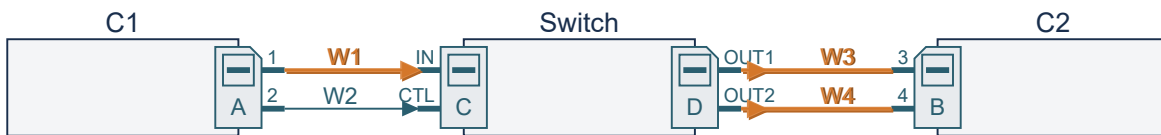


Figure 10. Schematic with highlighted electrically connected wires.

3.6.2. Connector-less Devices

`LComponent` objects are very similar to `Component` object, but do not have any connectors. The cavities are not grouped and can permute freely in order to minimize wire crossings. They support the same properties as `Component` objects. The same holds for their cavities. Also arcs are supported at `LComponent` objects.

Listing 13 shows an example how `LComponent` objects are created.

Listing 13. Creating an `LComponent` object

```

LComponent Sensor;
  Cavity 1;
  Cavity 2 | Type = in;
  Cavity 3 | Type = out;
  Cavity 4 | Type = halfdot;

```

3.6.3. Inliners

Inliners represent two cables that are connected to each other via connectors. Therefore they contain connectors and partnered cavities as shown in Listing 14. The keyword `Inliner` is used to create such an object. Connectors and cavities are added in the same way as to a `Component` object. However, we need to specify, which cavities on the two sides of the inliner are partnered (i. e., electrically connected). This is done using the keyword `Partner`.

In rather rare cases, an inliner can have more than one connector per side. In this case, the partner information alone is not sufficient, but we also have to tell EEvision which connectors are placed on the same side. Providing this information is the purpose of the connector type `anti`. All connectors on one of the two sides need to get this `Type=anti` flag as in the following example:

Listing 14. An inliner component with paired connectors and cavities

```

Inliner InI;
  Connector A | Type = female;
    Cavity (1:3);
  Connector B | Type = male anti;
    Cavity (1:3);
  Connector C;
    Cavity (1:2);
  Connector D | Type = anti;
    Cavity (1:2);
  Partner A.(1:3) = B.(1:3), C.(1:2) = D.(1:2);

```

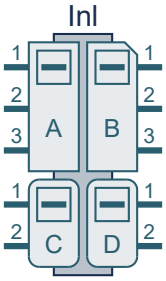


Figure 11. An inliner component with paired connectors and cavities; connectors B and D are on the same side as they are both flagged with `anti`.

3.6.4. Splices and Eyelets

Splices and Eyelets are used to establish an electrical connection between several wires. In EDML they are created using the keywords `Splice` and `Eyelet`, respectively. Both of them do not have connectors, but only cavities, as shown in Listing 15. Typically, both splices and eyelets get one cavity per wire that is connected to them. Cavity names of splices are not displayed in EEvision; cavity names of eyelets are. If this is not wanted, the name of eyelet cavities need to be explicitly removed as shown in Listing 15.

Listing 15. Creating splices and eyelets

```

Wire W1, W2;

Splice S1;
  Cavity 1, 2;
  Join 1 -> W1, 2 -> W2;
Eyelet E1;
  Cavity 1 | Name =;
  Join 1 -> W1;
LComponent Sensor;
  Cavity 1;
  Join 1 -> W2;

```



Figure 12. A splice and an eyelet

3.6.5. User-defined Component Symbols

EEvision offers the possibility to have user-defined symbol shapes for components. Such components have to be declared with the keyword `SVGComponent`. As the name of the keyword suggests, the outline is typically given by an SVG image that is displayed as the component's body. Then mainly the positions of the cavities need to be declared.

An `SVGComponent` (like an `LComponent`) does not have connectors.

Consider, e.g., the component definition in Listing 16.

Listing 16. Defining a component with a custom shape

EDML

```
SVGComponent R1 |
  Symdef = "DEF linewidth 1 fillcolor 1
    fpath -50 0 -50 130 50 130 50 0 -50 0
    imagedsp relay.svg -uc 0 0 148 130
    attrdsp @name -lc 0 -2 12";
  Cavity 1 | Symdef = "pin 1 inout.left -loc -58 32 -50 32
    pinattrdsp @name -ul -60 38 10";
  Cavity 2 | Symdef = "pin 2 inout.right -loc 50 32 58 32
    pinattrdsp @name -ur 60 38 10";
  Cavity 3 | Symdef = "pin 3 inout.left -loc -58 112 -50 112
    pinattrdsp @name -ul -60 118 10";
  Cavity 4 | Symdef = "pin 4 inout.right -loc 50 112 58 112
    pinattrdsp @name -ur 60 118 10";
```

The keyword `SVGComponent` is used here to declare a component with ID `R1`. Its shape is defined using the property `Symdef`. Its value needs to start with `DEF`, followed by a sequence of commands in Nlview's Symlib language. For instance, line 3 draws a filled polygonal path (`fpath`), the rectangle around the symbol. Line 4 loads an SVG image (`imagedsp`) with the name `relay.svg`, where the upper center (`-uc`) is placed at the coordinates (0, 0); the width and height are 148 and 130 pixels, respectively. The `Symdef` properties at the cavities define their positions and shapes (`pin`), aligned with the positions of the pins in the loaded SVG image, followed by the location where the cavity name is shown (`pinattrdsp`).

The resulting symbol is shown in Figure 13.

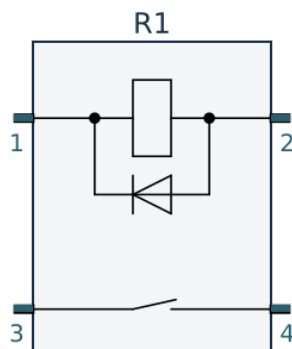


Figure 13. The custom symbol from the EDML definition in Listing 16. The interior of the rectangle is given by an SVG image.

For details about the Symlib language, we refer to the Nlview documentation.

3.6.6. Adding Labels to Components

EEvision allows to display the value of arbitrary attributes as labels at components, connectors, and cavities (at cavities only for components of type `Component` and `LComponent`). In the following example we use the value of the attribute `Desc` as the label value at components, connectors, and cavities. Note that the default configuration of EEvision in the file `eev.conf` needs to be adapted accordingly. The configuration keys `compAttrdsp` and `connAttrdsp` expect a list of attribute name; the values of the specified attributes are displayed, separated by newlines. In contrast, `cavAttrdsp` expects a single attribute name.

```
{  
  ...  
  compAttrdsp: ["Desc"],  
  connAttrdsp: ["Desc"],  
  cavAttrdsp: "Desc",  
  ...  
}
```

JSON

Then the following listing create two components with labels at the components, their connectors and cavities:

Listing 17. Components, connectors, and cavities with labels

```
Wire W(1:4);  
  
Component S741 | "Desc" = "Temperature Sensor";  
  Connector A | "Desc" = "Data Output";  
    Cavity 1 | "Desc" = "HOLD";  
    Cavity 2 | "Desc" = "Analog Out", Type = out;  
    Cavity 3 | "Desc" = "VCC +5V", Type = in;  
    Cavity 4 | "Desc" = "GND";  
  Join A.(1:4) -> W(1:4);  
  
Component A23 | "Desc" = "Temperature Controller";  
  Connector B | "Desc" = "Data Input";  
    Cavity 1 | "Desc" = "HOLD";  
    Cavity 2 | "Desc" = "Analog In", Type = in;  
    Cavity 3 | "Desc" = "VCC +5V", Type = out;  
    Cavity 4 | "Desc" = "GND";  
  Join B.(1:4) -> W(1:4);
```

EDML

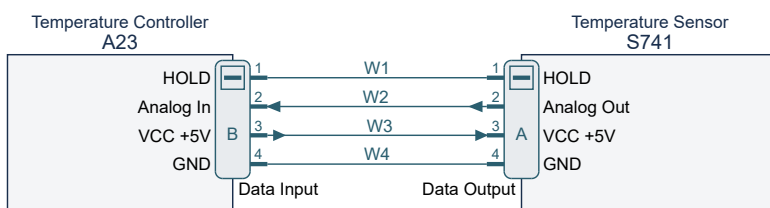


Figure 14. Components, connectors, and cavities with labels

Labels are possible at all types of components, the connectors of `Component`, `Inline`, and `HierComponent` objects (except invisible connectors), and the cavities of `Component` and `LComponent` objects.

3.7. Multicores

Multicores are wire bundles that are twisted and/or shielded. Multicores can be nested, e. g., two pairs of wires can be shielded together. Then these two bundles are twisted and shielded again. EDML allows to model such nested multicores easily.

Multicores are created using the `Multicore` keyword. What kind of multicore should be created is specified using the `Type` property. Supported values are `twisted`, `shielded`, and `twshielded`, which is the combination of twisted and shielded. For shielded multicores, additionally a shield wire can be specified using the `Shield` property. For nested multicores, the `Parent` property allows to assign a parent multicore. That implies that nested multicores are created starting from the outermost one. Wires are added to a multicore by listing their IDs in braces after the multicore ID and before the vertical bar.

An example of a nested multicore is shown in Listing 18; Figure 15 is the resulting schematic.

Table 9. Supported properties of multicores

Property	Values	Meaning	Repeat
Parent	<ID>	For nested multicores, this property specifies the ID of the parent (outer) multicore	no
Shield	<ID>	Specifies the shield wire of a <code>shielded</code> or <code>twshielded</code> multicore	no
Type	<code>shielded</code> , <code>twisted</code> , <code>twshielded</code>		no

Listing 18. Creating nested multicores

EDML

```
Wire W(1:8);
Component C1;
  Connector A;
    Cavity (1:4), (6:7);
    Cavity 5,8 | Type = halfdot;
  Join A.(1:8) -> W(1:8);
Component C2;
  Connector A;
    Cavity (1:4), (6:7);
    Cavity 5,8 | Type = halfdot;
  Join A.(1:8) -> W(1:8);

Multicore MC1 (W6, W7) | Type = shielded, Shield = W8;
Multicore MC2 (W1, W2) | Type = twisted, Parent = MC1;
Multicore MC3 (W3, W4) | Type = twshielded, Parent = MC1, Shield = W5;
```

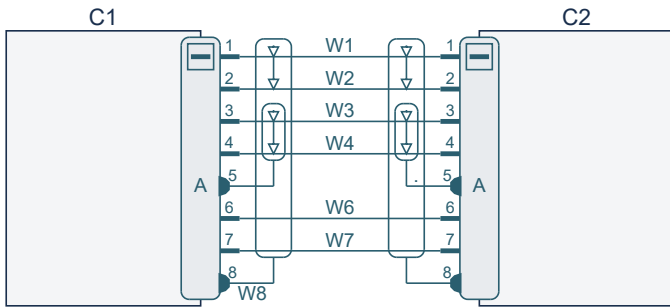


Figure 15. Nested multicores in EEvision

3.8. Modules

Modules are pre-defined groups of objects, which can easily be loaded and displayed in one step in EEvision. They are often used for showing all objects that form one function of a vehicle, all wires and connectors in a harness, or all wires carrying the same signal.

For this reason different types of modules are supported:

- Function modules, created using the keyword `Function` , typically contain all elements of a vehicle function.
- Harness modules are created using the keyword `Harness` . They contain normally all wires, cavities, connectors, splices and eyelets of a single harness.
- Signal modules (keyword: `Signal`) are supposed to contain all wires that carry the same signal.
- Bus modules (keyword: `DBus`) should contain all wires of the same bus.
- Finally, the keyword `Module` allows to define modules that do not fall in any of the other categories.

Module information is also used for automated document generation and for smart system debugging with EEvision.

For example, the EEvision DocGen enterprise plugin uses the information in the function modules and harness modules to automatically generate PDF or HTML data books that contain schematic diagrams and BOM information for all functions and harnesses in a particular vehicle, aircraft, or machine.

For smart system debug, function modules or harness modules can also be very useful. Using the EEvision debugging cockpit, engineers can quickly get an overview about all the functions and harnesses in their system. The EEvision module navigator plugin (`modnav`) allows even to see if certain system functions are interfering each other, or if harness connections are properly aligned.

All classes of modules (except `Always` and `Config` modules, see below) follow the same syntax pattern:

```
Keyword ID (object ID, object ID, ...) | Properties, Attributes;
```

TEXT

Listing 19. Creating a function module

```

Wire W(1:8);
Component C1;
    Connector A;
        Cavity (1:4);
    Connector B;
        Cavity (1:4);
    Join A.(1:4) -> W(1:4), B.(1:4) -> W(5:8);
Component C2;
    Connector C;
        Cavity (1:4);
    Connector D;
        Cavity (1:4);
    Join C.(1:4) -> W(1:4), D.(1:4) -> W(5:8);

Function TC
( W(1:3), C1, C1.A, C1.A.(1:3), C2, C2.C, C2.C.(1:3))
| Name = "Temperature Control", "Location" = "Front Left";

```

Currently, there is only one module-specific property, namely `Option=autocomplete`. It is mainly used for function modules and allows to specify a function by just listing all `Component`, `LComponent`, and `SVGComponent` cavities that are relevant for the function. EEvision then adds all wires, splices, eyelets, and inliners that are needed to electrically connect these selected cavities.

In the example above, we could simplify the definition of the `Function TC` as follows:

```

Function TC (C1.A.(1:3), C2.C.(1:3))
| Name = "Temperature Control", Option = autocomplete, "Location" = "Front Left";

```

Table 10. Supported properties of modules

Property	Values	Meaning	Repeat
Option	autocomplete	The module only contains the Component/LComponent/SVGComponent cavities that are relevant for the module. The option <code>autocomplete</code> tells EEvision that the wires, inliners, splices, and eyelets need to be added automatically.	no

3.8.1. Shorthand Notation

It is a very common situation that a whole component together with all of its connectors and cavities has to be added to a module. Listing all connectors and cavities explicitly is both time consuming and error prone. For this case, EDML offers a shorthand notation.

Consider the following example:

```

Wire W1, W2;
Component Sensor;
    Connector A;
        Cavity x,y,z;
    Connector B;
        Cavity a,b,c;
    Join A.x -> W1, B.a -> W2;
Component Actor;
    Connector X;
        Cavity 1,2;
    Join X.1 -> W1, X.2 -> W2;

Function F1 (
    Sensor,
    Sensor.A, Sensor.A.x, Sensor.A.y, Sensor.A.z
    Sensor.B, Sensor.B.a, Sensor.B.b, Sensor.B.c,
    W1, W2,
    Actor, Actor.X, Actor.X.1, Actor.X.2
);

```

It creates a function module that contains the components `Sensor` and `Actor` together with their connectors and cavities as well as both wires `W1` and `W2`.

In this case we could also write

```
Function F1 (Sensor+, W1, W2, Actor+);
```

The ` following a component ID means that we want to add the component, all of its connectors, and all cavities to the module. Similarly, `Sensor.A would add the connector A of Sensor with all its cavities to the module. If we add ` as a suffix to a wire ID like `W1, the wire, all cavities it is connected to plus their connector and component are added. In our example, W1+ is equivalent to W1, Sensor.A.x, Sensor.A, Sensor, Actor.X.1, Actor.X, Actor.

This shorthand notation is possible for all types of modules. Duplicate objects are only added once.

3.9. Configurations

EEvision allows to store a complete 150% vehicle model in a database, i.e., a model that contains all possible configurations of a vehicle at once. If the actual configuration of a vehicle is known, EEvision is able to create a 100% model of that vehicle automatically.

For this to work, the possible configurations have to be stored in the database using two special kinds of modules:

- Always modules contain those objects that are part of *all* possible configurations.
- In contrast, Config modules encompass all those objects that are only present in *some* vehicle configurations, e.g., depending on the available optional equipment.

The syntax for creating these modules is similar for `Always` and `Config` modules. Listing 20 shows an example with one `Always` and two `Config` modules that are used to configure an inliner component `Inl1` and a wire `w1`.

Listing 20. Creating configurations in EDML

EDML

```
Wire W1;

Inliner Inl1;
  Connector A | Type = female;
    Cavity 1,2,3;
  Connector B | Type = anti male;
    Cavity 1,2,3;
  Connector C | Type = anti male;
    Cavity 1,2,3;

Always always_mod;
  Objects W1, Inl1, Inl1.A+;

Config c1 | Expr = "Heat & !Radio";
  Objects Inl1.B, Inl1.B.(1:3);
  Attributes Inl1 | "Usage" = "Engine control";
  Join Inl1.B.1 -> W1;
  Partner Inl1.A.(1:3) = Inl1.B.(1:3);

Config c2 | Expr = "!Heat | Radio";
  Objects Inl1.C+;
  Attributes Inl1 | "Usage" = "Entertainment system";
  Join Inl1.C.2 -> W1;
  Partner Inl1.A.(1:3) = Inl1.C.(1:3);
```

Lines 12 and 13 create an `Always` module that contains the wire `w1`, inliner component `Inl1`, its connector `A` with the cavities `1`, `2`, and `3`. They are contained in all possible configurations of the vehicle.

Additionally, two `Config` modules `c1` and `c2` are available. They have a property `Expr` whose value is a Boolean expression that activates its `Config` module whenever the expression evaluates to true. The variables in these expressions (here: `Heat` and `Radio`) can, e.g., refer to optional equipment. The expression `Heat & !Radio` activates `c1` when `Heat` is true and `Radio` is false. These expressions are optional; one can also activate the `Config` modules directly. For more details on activating a configuration, we refer to the EEvision documentation.

For adding data to a `Config` module, one uses the statements `Objects`, `Attributes`, `Join`, and `Partner`:

- `Objects` lists all those objects that should be available in the database when the configuration is applied. In our example, if `c1` is activated, the connector `B` of `Inl1` is added together with its three cavities `1`, `2`, and `3`. Note that the shorthand notation introduced in Section 3.8.1 can also be used for these object lists. So, we could abbreviate the object list of `c1` to `Objects Inl1.B+;`.
- Additionally, in line 17, the inliner component `Inl1` gets an attribute with name `Usage` and value `Engine control`.

- Line 18 tells the compiler that wire `w1` should be connected to cavity `1` of connector `B` at `In11` if the configuration `c1` is applied.
- Lastly, in Line 19, the cavities of connector `B` are partnered with the corresponding cavities of connector `A`. The second `Config` module `c2` is defined similarly.

For `Always` modules, only `Objects` statements are supported. Configurable attributes, wire-cavity connections, and cavity partner relations in `Always` modules are equivalent to adding them directly to the database.

3.10. Hierarchical Models

EEvision supports hierarchical models where certain kinds of components can contain other components and wires. For this, EDML provides two different types of hierarchical components:

- Components of type `HierBox` are simple rectangular boxes without connectors or cavities. They can contain components, wires, multicores, and other hierarchical components. `HierBoxes` are mainly used for grouping objects.
- The component type `HierComponent` denotes hierarchical ECU components, which have connectors and cavities both on the outside and on the inside. They are typically used for closed sub-systems that are connected to the main harnesses via a connector. A typical example is an outside mirror of a vehicle: It contains several components (motors, control units, cables) and is connected to the main vehicle harnesses via one or more connectors.

Both types of components can be nested arbitrarily deeply. EEvision allows to fold and unfold hierarchical components to hide or show their contents. The same dataset can contain both kinds of hierarchical components.

In the following, we provide more information about modeling hierarchy in EDML.

3.10.1. Hierarchical Boxes (`HierBox`)

A hierarchical box is created using the keyword `HierBox`. It has neither connectors nor cavities; attributes and properties can be specified as usual. The content of the box follows after the keyword `HierContent` in curly braces:

```
HierBox H1 | "attr" = "value", ...;
HierContent {
    // Content of the HierBox
}
```

EDML

The properties `Name`, `Href`, `Image`, `Video`, and `Color` are supported. The latter is used to color the body and/or the border of the box; by default, `HierBoxes` have a transparent background.

Wires cannot connect to a `HierBox` since a `HierBox` does not have any cavities. However, wires may cross the boundaries of a `HierBox`, i. e., the same wire can connect to cavities outside of a `HierBox` and within a `HierBox`. For this to work, the wire has to be declared on the deepest hierarchy level that contains all cavities the wire is connected to. The following example illustrates how to create a hierarchical design using `HierBox` objects.

The following EDML code creates two nested HierBoxes and three components, which are connected via two wires. The HierBox H2 gets a light yellow ("lightgoldenrodyellow") background color and a dark red border. The resulting schematic is shown in Figure 16.

Listing 21. Creating HierBox components in EDML

EDML

```
Wire W1;

HierBox H1;
  HierContent {
    Wire W2;

    Component C1;
    Connector A;
    Cavity 1, 2;
    Join A.1 -> W1, A.2 -> W2;

    HierBox H2 | Color="lightgoldenrodyellow darkred";
    HierContent {
      Component C2;
      Connector B;
      Cavity x1;
      Join B.x1 -> W2;
    }
  }

LComponent C3;
  Cavity 1;
  Join 1 -> W1;
```

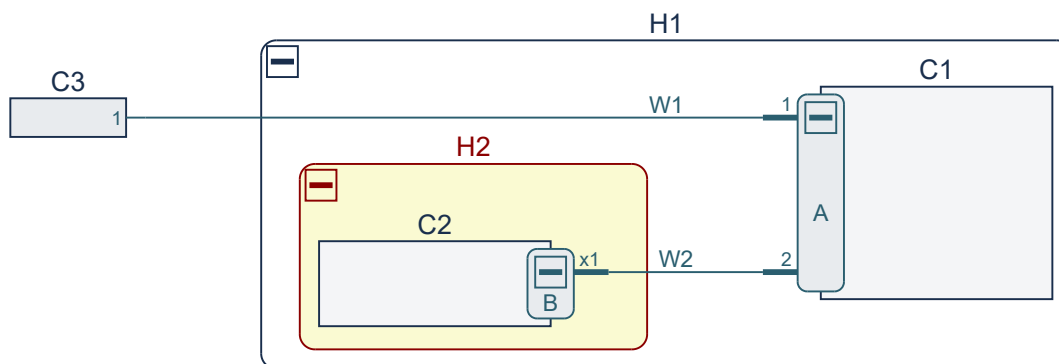


Figure 16. Schematic resulting from the EDML code in Listing 21.

The wire **w1** has to be declared on the outermost level as it connects a cavity on the outermost level with a cavity within **H1**. On the other hand, wire **w2** is declared within **H1** as it connects only cavities within **H1**.

3.10.2. Hierarchical ECUs (HierComponent)

Compared to the simple HierBox components, the structure of hierarchical ECUs is more similar to **Inliner** components with partnered connectors/cavities on the outside and the inside. Connectors whose type includes the flag **anti** are inside the hierarchical component, connectors without that flag on the outside. The **Partner**

statement is used to associate corresponding cavities on the outside and inside.

The contents of a `HierComponent` are declared within a `HierContent` block, similar to `HierBoxes`. Besides the components, wires, and multicores that are declared within the `HierContent` block, the cavities of those connectors of the parent `HierComponent` that are declared with the `anti` flag can be accessed from within the `HierContent` block as part of a `Join` statement.

Wires may **not** cross the hierarchy border of `HierComponents`, i. e., a wire may not connect to cavities on the outside and the inside of a `HierComponent`. The cavities of the `HierComponent` that belong to a connector with an `anti` flag are on the inside, the other ones on the outside.

NOTE

Due to technical limitations in EEvision rendering engine, the inner connectors are currently shown as detached from the `HierComponent`. They can either be matched by name or by clicking on the "[P]" button on the connector. Then the corresponding connector on the other side will be highlighted. Removing this limitation is planned for future versions of EEvision.

The following example illustrates how to create a `HierComponent`.

The following EDML code creates a hierarchical design with a `HierComponent`:

Listing 22. Creating a `HierComponent` in EDML

EDML

```
Wire W(1:3);

Component C1;
  Connector A;
    Cavity (1:3);
  Join A.(1:3) -> W(1:3);

HierComponent H1;
  Connector B | Type = male;
    Cavity (1:3);
  Connector C | Type = female anti;
    Cavity (1:3);
  Partner B.(1:3) = C.(1:3);
  Join B.(1:3) -> W(1:3);

  HierContent {
    Wire U(1:3);
    Join C.(1:3) -> U(1:3);

    Component C2;
      Connector D;
        Cavity (1:3);
      Join D.(1:3) -> U(1:3);
  }
```

The Component C1, the HierComponent H1, and the wires W1 to W3 belong to the root level of the hierarchy. The connector B of H1 is on the outside of the hierarchy because it does not have the `Type` flag `anti`. The other connector C of H1, in contrast, belongs to the inside of the hierarchy because its `Type` contains the flag `anti`.

The resulting schematic looks as follows:

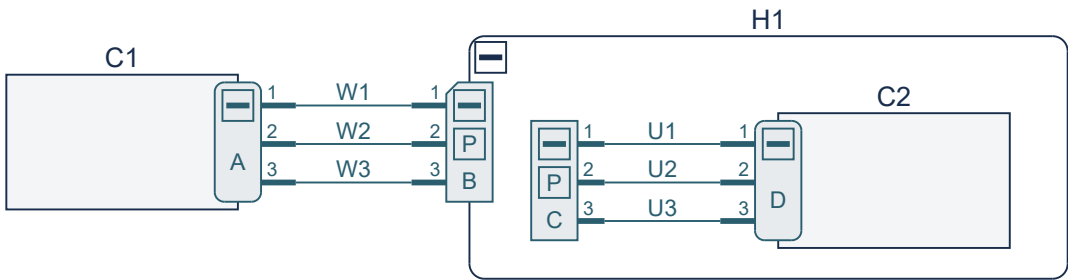


Figure 17. Schematic resulting from the EDML code in Listing 22.

3.11. Internationalization

EEvision supports a mechanism to display the schematics in different languages, provided that the EDB database contains the language information. This mechanism is based on appropriately named attributes and therefore also supported in EDML.

Making attributes language-specific is done by adding a suffix in round braces to the attribute name like in `Länge(DE)`. Typically, the language code is used for the suffix, i. e., all attributes with the suffix `(DE)` are considered the German variants. The attributes with only the suffix as their name, e. g., `(DE)`, are considered to be the translations of the object's `Name` property.

Listing 23 shows an example of an EDML file that adds the languages English (EN), German (DE), and Greek (GR) to a simple model. The resulting schematics after activating the three languages are displayed in Figure 18.

Listing 23. Creating a multi-language model

```
Wire W1 | Name = "Power supply", Type = power,
  "(EN)" = "Power supply",
  "(DE)" = "Stromversorgung",
  "(GR)" = "Παροχή ρεύματος",
  "Length(EN)" = "10 in",
  "Länge(DE)" = "25.4 cm",
  "Μήκος(GR)" = "25.4 cm";
```

```
Component S1 | Name = "Temperature Sensor",
  "(EN)" = "Temperature Sensor",
  "(DE)" = "Temperatursensor",
  "(GR)" = "Αισθητήρας θερμοκρασίας";
```

```
Connector A;
  Cavity 1;
Join A.1 -> W1;
```

```
Component Battery | "(EN)" = "Battery", "(DE)" = "Batterie", "(GR)" = "Μπαταρία";
Connector A;
  Cavity 1;
Join A.1 -> W1;
```

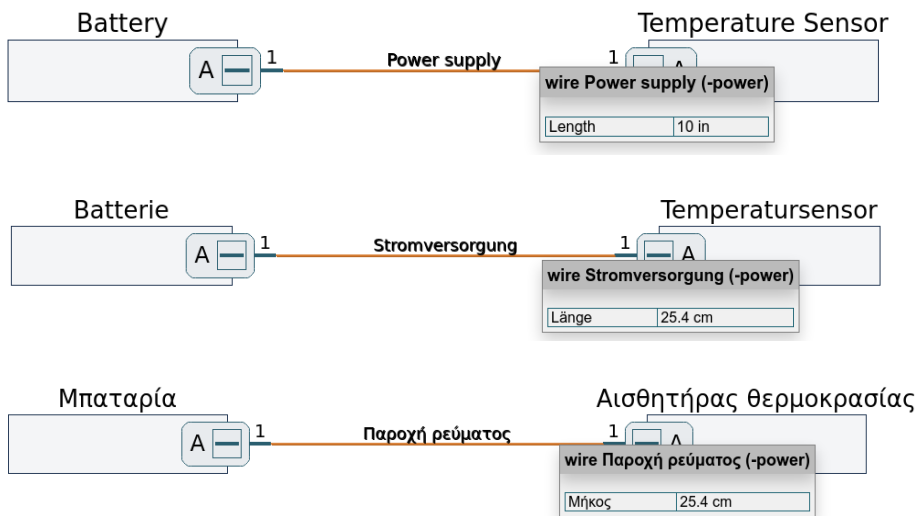


Figure 18. Schematics with different languages (here from top to bottom: English, German, and Greek)

We refer to the [EEvision documentation](#) for the details of the internationalization process.

3.12. Component Libraries

EDML component libraries allow to reuse component definitions in different projects and to instantiate them multiple times within a design. The possibility to have multiple identical symbols (with different names) is particularly useful for components like fuses, relays, sensors etc.

The power of component libraries comes from three features:

- An `#include` directive allows to split a design into several files. This way parts of a design can be shared between projects.

- A `Define` statement allows to create own component types that can later be instantiated.
- Such component definitions can be equipped with parameters that are set when creating components. These parameters allow to set attributes and properties to instance-specific values.

In the following we provide more details on these features.

3.12.1. Multiple Files per Project

In order to keep models readable and well structured, EDML allows to split a model into several files using the `#include` directive. The effect of an `#include` directive is that the EDML compiler replaces the line

```
#include "filename.edml"
```

with the contents of the referenced EDML file (which can also contain `#include` statements as long as there are no cycles). This mechanism is particularly handy combined with the possibility to define new component types as we have already seen in the introductory example. While arbitrary parts of the model can be moved to their own file, the include mechanism is typically used to include definitions of new component types, as described in the following section.

3.12.2. Component Definitions

EDML not only provides the built-in component type `Component`, `LComponent`, `SVGComponent`, `HierComponent`, `Splice`, and `Eyelet`, but also allows users to define their own component types as refinements of the built-in ones.

For instance, consider the following EDML code:

Listing 24. Defining a new component type

```
Define {
  Component Sensor | Color = "yellow";
    Connector A | Type = male;
      Cavity 1 | Type = in;
      Cavity 2 | Type = out;
    Connector B | Type = male;
      Cavity 1;
      Cavity 2;
    Arc internal_arc (A.1, B.2);
}

Wire W(1:4);

Sensor S1;
  Join A.(1:2) -> W(1:2);
  Join B.(1:2) -> W(3:4);
Sensor S2;
  Join A.1 -> W2, A.2 -> W1;
  Join B.(1:2) -> W(3:4);
```

EDML

It defines a new component type `Sensor`, which is a refinement of the built-in type `Component`. As we can see in Lines 13 and 16, we can create new components using the newly defined type `Sensor`. In this case, we obtain two components named `S1` and `S2`, both with the property `Color="yellow"` and two connectors `A` and `B`, which are both `male` and have two cavities `1` and `2`. Connecting wires to these components works in the same way as with built-in types.

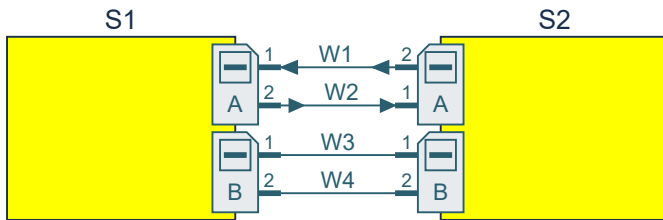


Figure 19. Two identical instances of a `Sensor` component

So the effect of a `Define` statement is that the defined type is replaced by the compiler with its definition; only the ID is replaced with the ID of the defined object.

3.12.3. Parameterization

If we define new component types as described in the previous section, all instances of these types share all properties and attributes. However, in practice some of these properties and attributes need to get instance-specific values. EDML allows to add parameters to `Define` statements, which can be set when a user-defined component type is instantiated.

Parameters need to be declared in the `Define` statement after the name of the new type using the keyword `Parameter`, followed by a list of parameter names. The syntax of parameter names is the same as the syntax of object IDs. We can assign default values to parameters.

Listing 25 shows an example of a type definition that contains the parameters `col`, `loc`, `harA`, and `harB`; `col`, `harA`, and `harB` have default values: the default value of `col` is `"yellow"`, the one of `harA` and `harB` is empty. Parameter without default values need to be specified when instantiating the component type. In contrast, parameters with default value can be omitted; in this case they take their default value. A special case of a value is the empty value (in the example: `harA=`, `harB=`). If a parameter has an empty value, the property or attribute that is assigned this value is omitted from the created object.

The declared parameters can be used in the component definition as the value of attributes and those properties that expect a string as their value; they can not only provide string values in the component itself, but also in its connectors and cavities, i.e., for everything within the curly braces of the `Define` statement. For obtaining the value of a parameter, its name is prefixed with a dollar sign (`$col`, `$loc`, etc.). Writing `Color=$col` assigns the value of the `col` parameter to the property `Color`.

Listing 25. Adding parameters to component type definitions

```

Define {
  Component Sensor Parameter col = "yellow", loc, harA =, harB =
    | Color = $col, "Location" = $loc;
    Connector A | "Harness" = $harA;
    Cavity 1 | Type = in;
    Cavity 2 | Type = out;
    Connector B | Type = male, "Harness" = $harB;
    Cavity 1;
    Cavity 2;
}

Wire W(1:4);

Sensor S1 | col = "orange blue", loc = "Front Left",
  harA = "H01-02-17", harB = "H01-02-18";
Join A.(1:2) -> W(1:2);
Join B.(1:2) -> W(3:4);

Sensor S2 | loc = "Front Right", harA = "H01-02-17";
Join A.1 -> W2, A.2 -> W1;
Join B.(1:2) -> W(3:4);

```

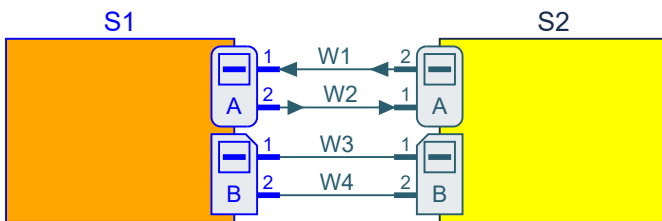


Figure 20. Schematics generated from the EDML in Listing 25

Parameter assignment happens when a `Sensor` component is created (see Lines 14 and 19 of Listing 25). The syntax is the same as for properties: after the vertical bar `|`, the parameters are assigned; the parameter name is specified without quotation marks, followed by an equality sign and the parameter value. Note that parameter names must be different from the property names.

4. Tool Support

4.1. The edm12edb Compiler

While EDML provides an intuitive format that is easy to write and understand by humans, EEvision’s internal database format EDB is optimized for automatic processing and efficient access. For translating EDML files into the internal EDB format, the EEvision PRO package contains the `edm12edb` compiler.

This compiler not only does the mere translation from EDML to EDB, but also assists the user in debugging the design by optionally adding references from all created objects to their corresponding location in the EDML file, by not only performing syntax checking but also pointing the user to common mistakes and semantic errors.

4.1.1. Using the Compiler

The executable of the compiler is named `edm12edb` (Linux/Unix) and `edm12edb.exe` (Windows), respectively. It can be found in the directory `edb/` of the EEvision distribution. Note that it requires an EEvision PRO license for running.

The command for calling the compiler looks as follows:

```
edm12edb[.exe] ?options? <design.edml>
```

where `<design.edml>` is the name of the input file. The compiler offers the following options:

Option	Description
<code>-help</code>	Prints some help text that lists the available options and exits.
<code>-o <out.edb></code>	Specifies the name of the EDB file that is generated by the compiler.
<code>-extern <in.edb></code>	If the EDML file uses the “extern” keyword (see Chapter 5), one needs to specify an existing EDB file to which the “extern” declarations refer.
<code>-srcref <base></code>	Adds source references to the generated EDB for debugging the design. Removes trailing base from source file names (to save some space).
<code>-dump <out.dump></code>	Dump file (for debugging the compiler)
<code>-log <out.log></code>	logfile (for debugging the compiler)

The `-srcref` option deserves particular attention. It is used to facilitate debugging an electrical design in EEvision. If this option is specified when calling the compiler, the generated EDB file contains for each object the file name and line number where the object is created in the EDML model. This information makes it easy to identify the location in the EDML file that needs to be adjusted when errors in the model are found.

The option `-srcref` is used only during development. When the EDB file is released after having passed all checks, this option should be omitted.

4.1.2. Licensing

The `edm12edb` compiler is an optional component of EEvision, which requires a separate license for running. Please refer to the EEvision documentation for licensing details or contact the EEvision support team of Altair Engineering.

4.2. The Visual Studio Code Extension

```
(  
h  
tt  
p  
s:  
//  
c  
o  
d  
e  
.  
vi  
s  
u  
a  
ls  
t  
u  
d  
i  
o  
.c  
o  
m
```

For the popular source code editor [Visual Studio Code](#) ¹⁾, we provide an extension package to simplify editing EDML files. The extension provides features like:

- syntax highlighting,
- automatically closing brackets,
- shortcuts `Ctrl + Shift + 7` and `Ctrl + Shift + A` for toggling line and block comments, respectively.

4.2.1. Installation

The extension can be found in the EEvision package in the directory `vscode`. Its file name is `edm1.vsix`. There are two possible ways to install it in Visual Studio Code:

Either on the command line by executing the following command in the directory that contains the extension file:

```
code --install-extension edml.vsix
```

BASH

Alternatively, you can use the `Install from VSIX` command in the Extensions view command dropdown, or the `Extensions: Install from VSIX` command in the Command Palette. Then select the `edml.vsix` file.

(
h
tt
p
s:
/
c
o
d
e
.
vi
s
u
a
ls
t
u
d
i
o
.c
o
m
/
d
o
c
s
/
e
d
it
o
r
/
e
x
t
e
n
si
o
n
-
m
a
r
k

e
t
p
l
a
c
e

_i
n
s
t
a
ll
-
fr
o
m
-
a
-
v
si
x

For more information, see the [official documentation](#)) on installing packaged extensions in Visual Studio Code.

5. Extending and Modifying Existing Models

Apart from modeling complete electrical systems, EDML can also be used to extend an existing EDB database, e. g., by adding further components and wires as well as adding new connectors and cavities to existing components. Attributes and some of the properties can not only be added to existing objects, but also be modified. For this purpose, EDML supports the `extern` keyword and some variants thereof.

5.1. Describing Extensions

For extending an EDB database, one typically has to refer to existing objects in that database for modifying them. For this purpose, it must be declared in the EDML file first with the keyword `extern` (or one of its variants that are described below).

For instance, assume there is a component with the name `Temperature Sensor` that needs to get an additional connector `C` with cavities `x1` , `x2` , and `x3` . We also want to add an attribute `PartNo` with value `A3421-B23` to the component and change the value of the attribute `Location` to `right` .

The EDML file that makes these changes to `Temperature Sensor` looks as follows:

```
extern Component TS | Name == "Temperature Sensor",
    "PartNo" = "A3421-B23", "Location" = "right";
    Connector C;
    Cavity x(1:3);
```

The EDML description above references the `Temperature Sensor` and assigns to it the identifier `TS`. The identifier `TS` (not the component name) is then used to refer to this component within the EDML file. Note the difference between `==` and `=`. The former is used to select an object that satisfies the *selection condition*; the latter assigns a new value to a property or an attribute.

Only those objects that need to be referenced — for instance, for adding extensions or modifying attributes — need to be declared. If `Temperature Sensor` has, e. g., further connectors and cavities that do not need to be extended, they can be omitted in the declaration.

The `extern` keyword can only be applied to built-in object types like `Component`, `Connector`, `Cavity`, `Wire` etc. It is not possible to combine it with user-defined object types. The following example throws an error:

```
Define {
    Component Sensor;
    Connector A;
    Cavity 1,2;
}

extern Sensor S1;
```

Instead it must be referenced with its built-in type:

```
extern Component S1;
```

5.2. Calling the Compiler

Assume, you have an EDB database named `model.edb` that you want to enrich with the additional objects defined in a file `extension.edml`. Then the `edm12edb` compiler is called as follows:

```
edm12edb[.exe] -extern model.edb -o model_extended.edb extension.edml
```

The result is an extended EDB file named `model_extended.edb` that not only contains the contents of `model.edb`, but also the extensions described in the file `extensions.edml`.

5.3. Selecting Multiple Objects

The `extern` keyword requires that the object can be identified uniquely. For a connector, it is sufficient that it can be identified uniquely among the connectors of the same component (the case of cavities is similar). If there is not exactly one object that matches the selection condition(s), the `edm12edb` compiler will issue an error.

Sometimes there might be the need to find all objects that match a selection condition and treat them in a uniform way. For instance, we might want to select all wires whose `signal` attribute has the value `+12V` and add them to a `Signal` module named `Power`.

For this purpose, EDML offers two variants of the `extern` keyword:

- With `externN`, an arbitrarily long list of objects is bound to an ID. It is allowed that no object, one object, or many objects match.
- The keyword `extern0` is similar to `extern`, but does not generate an error when no matching object is found. However, an error is generated when more than one object matches.

The following code snippet adds all wires with the attribute `signal` and value `+12V` to a new `Signal` module named `Power`. Additionally all these wires get the color red. If no such wire exists, the `Signal` module stays empty.

```
externN Wire PWR | "signal" == "+12V", Color = "red";
Signal Power (PWR);
```

EDML

IDs that are declared using `extern`, `externN` or `extern0` represent a list of objects of the same kind (like components, wires, ...). All operations involving such list-valued IDs are applied to all elements of the list. The keywords, however, impose restrictions on the list length.

Consider the following code:

```
externN Wire PWR | "signal" == "+12V";
Component C1;
  Connector A;
    Cavity 1;
      Join A.1 -> PWR;
```

EDML

It selects *all* wires with signal `+12V` and connects *all* of them to cavity `1` at connector `A` of the newly created component `C1`.

Some operations are only possible if the list of objects represented by an ID contains at most one object, e. g., partnering cavities or declaring a multicore as the parent of another multicore. At these places, object IDs declared with `externN` are not allowed.

Table 11. Overview of the keywords for referencing existing objects in an EDB

Keyword	Allowed number of matched objects
---------	-----------------------------------

Keyword	Allowed number of matched objects
extern	1
extern0	none or 1
externN	arbitrarily many (including none)

5.4. Selection Conditions

For selecting objects, both property values and attributes can be used. For instance:

The statement

```
extern Wire W1 | Name == "K1. 31", Color == "red", "PartNo" == "72359";
```

EDML

selects the wire whose name is `K1. 31`, which has a red color marker and a part number of `72359`. Since multiple selection conditions are specified, objects need to satisfy *all* of them to get selected. Since `extern` is used (and not `extern0` or `externN`), there must be exactly one matching wire.

The same attribute or property can be used as part of the selection condition and in an assignment within the statement. The order of the entries after the `|` is irrelevant. First all selection conditions are evaluated to determine the list of matching objects; then all assignments are made to the objects in the list.

If no selection condition is explicitly given, the ID is used as the object name. That means:

```
extern Wire Wground;
```

EDML

is equivalent to

```
extern Wire Wground | Name == "Wground";
```

EDML

Besides the comparison with `==`, which checks for equality, also `!=` is supported, which checks for inequality.

```
externN Wire W1 | Name == "K1. 31", Color != "red", "PartNo" != "72359";
```

EDML

This EDML statement selects all wires with the name `K1. 31`, whose color marker is *not* red and whose part number is *not* `72359`. They are then accessible with the ID `w1`.

The third supported comparison operator is `~=` . It does not compare the property or attribute value with a

(
h
tt
p
s:
//
e
n
.
w
i
k
i
p
e
d
i
a
.
o
r
g
/
w
i
k
i/
G
l
o
b
_
p
r
o
g
r
a
m
i
n
g

constant string or value, but checks if it matches the glob pattern)) on the right-hand side of `~=` .

Similarly, `!~=` can be used to select objects which do not match the given glob pattern.

This statement takes all wires that contain the string "gnd" (case-insensitive) in their name and creates an attribute "Signal" with value "ground":

```
externN Wire GND | Name ~= "[gG][nN][dD]*", "Signal" = "ground";
```

EDML

5.5. Changeable Properties

In an `extern` / `externN` / `extern0` statement all attributes of an object can be modified. However, not all properties can be changed. For instance, it is not always allowed to change the type of an object. The properties that may be modified in such an `extern` statement are:

- Color
- Ecfile
- Expr
- Href
- Image
- Imagedsp
- Name
- Subtype
- Style
- Symdef
- Video

If an `extern` object already has multiple attributes with the same name, setting an attribute with that name in the EDML file replaces all attributes of that name with the newly declared ones.

As an example assume that the EDB contains a component `c1` with two attributes `"Attr" = "V1"` and `"Attr" = "V2"` with the same name. We declare it in the EDML file as follows:

```
extern Component C1 | "Attr" = "V3", "Attr" = "V4", "Attr" = "V5";
```

EDML

The attribute declaration overrides both existing attributes named `Attr` with the three newly declared attributes with that name. Consequently, afterwards the component `C1` has three attributes, which have the name `Attr` and the values `v3`, `v4`, and `v5`, respectively.

To remove an attribute or property, specify the attribute/property name, but no value.

The following line references a component with name "Temperature Sensor" from an existing database and removes both the attribute "PartNo" and its Color property.

```
extern Component C | Name == "Temperature Sensor", "PartNo" =, Color =;
```

EDML

5.6. Example

Listing 26 shows a larger example that illustrates the possibilities for extending an existing model.

Listing 26. A larger example for extending an existing EDB

```
externN Wire W12V | "signal" == "+12V", Type == power; // (1)
Wire W1 | Type = power, "signal" = "+12V";           // (2)

extern Component TS |
    Name == "Temperature Sensor",
    "PartNo" = "3948B76";                             // (3)
extern Connector A;                                     // (4)
    extern Cavity 1;
    Cavity 5;                                           // (5)
Connector B;                                           // (6)
    Cavity x(1:3);
Arc R1 (A.1, B.x2);                                   // (7)
Join A.5 -> W1;

extern Inliner X23;
    extern Connector P;
        extern0 Cavity x(1:3);                         // (8)
Connector J | Type = female;                         // (9)
    Cavity (1:3);
Partner P.x(1:3) = J.(1:3);
Join J.1 -> W1;                                         // (10)

extern Multicore MC17 (W1);                            // (11)

extern Function F1 (W1) | Name == "Air Conditioning"; // (12)
Signal sig_pwr (W1, W12V) | Name = "+12V";           // (13)
```

EDML

1. selects all power wires with name "W12V" and signal +12V from the EDB, if any exist, and assigns them to the ID W12V .
2. adds a new power wire to the database whose name and identifier is W1 . Its signal attribute is set to +12V . Adding new objects works exactly as described in the previous sections of this document.
3. finds an existing component with the name Temperature Sensor in an EDB, assigns the identifier TS to it and creates a new attribute PartNo (or updates an existing attribute with that name).
4. searches for the existing connector A of component TS and for a cavity with name 1 at this connector.
5. extends the existing connector A of component Temperature Sensor by another cavity 5 .

6. adds an additional connector `B` with cavities `x1`, `x2`, and `x3` to this component. Cavity `5` of connector `A` is connected to the new wire.
7. This line adds a new Arc to the component that models an component-internal electrical connection between cavity `1` of connector `A` and cavity `x2` of connector `B`.
8. We check for cavities `x1`, `x2`, and `x3` at connector `P`. If they exist (and are unique at their connector), they are bound to the ID `x1`, `x2`, and `x3`, respectively.
9. The inliner `X23`, which consists only of one unpaired connector `P` with three cavities in the database, gets a partner connector `J`;
10. joins the existing cavity `1` at connector `J` with the newly create wire `w1`.
11. adds the new wire `w1` to the existing multicore `MC17`.
12. adds the same wire `w1` also to the function module `Air Conditioning`.
13. create a new signal module with ID `sig_pwr` and name `+12V`. It contains the new wire `w1` and the existing wires that have been bound to the ID `w12v`.

5.7. Order of Statements

If `extern` statements are mixed with statements that create objects, the order of the statements matters. This effect is inherent to the way how an EDML file is processed: First the existing EDB is loaded into memory. Then the statements are processed from top to bottom. New objects are created within the loaded EDB. `extern` statements are applied to the current collection of object.

Consider the following two EDML descriptions:

```
Wire W1 | "signal" = "+12V";  
extern0 Wire W12V | "signal" == "+12V";
```

EDML

and

```
extern0 Wire W12V | "signal" == "+12V";  
Wire W1 | "signal" = "+12V";
```

EDML

In the first case, the loaded EDB is first extended with an additional wire with name `w1` and `"signal" = "+12V"`. Then all wires with that signal are selected and bound to the ID `w12v`. This list includes the newly created wire.

In contrast, in the second example, first the wires with `"signal" = "+12V"` are selected and bound to the ID `w12v`. After that, an additional wire is created. Therefore it is not contained in the object list of `w12v`.

5.8. Changing Component Types

For some kinds of components, it is possible to change their type when referencing them in an external EDB using the `extern` keyword.

The following code converts a splice into an eyelet and vice-versa. Additionally, an LComponent is turned into an SVGComponent and extended with a symbol description:

```
extern Splice S1 as Eyelet | Name == "SP 12";
extern Eyelet E1 as Splice | Name == "EY 42";
extern LComponent Sensor as SVGComponent |
    Symdef = " ... ";
```

EDML

Note that not all kinds of components can be converted into each other. Currently supported are:

- `Splice` → `Eyelet`
- `Eyelet` → `Splice`
- `LComponent` → `SVGComponent` (see Section 3.6.5)
- `SVGComponent` → `LComponent`
- `Component` → `HierComponent` (see Section 3.10.2)
- `Component` → `Inliner`

Appendix A: EDML Keywords

A.1. Object Declaration

Keyword	Used within the scope of
Always	<i>global</i>
Arc	Component , LComponent , SVGComponent
Attributes	<i>global</i> , Config
Cavity	LComponent , Splice , Eyelet , SVGComponent , Connector
Component	<i>global</i> , Define , HierContent
Config	<i>global</i>
Connector	Component , Inliner , HierComponent
DBus	<i>global</i>
Define	<i>global</i>
extern	Arc , Component , LComponent , SVGComponent , HierComponent , Inliner , Splice , Eyelet , Wire , Multicore , Function , Harness , Signal , DBus , Module , Connector , Cavity
externN	same as extern
extern0	same as extern
Eyelet	<i>global</i> , Define , HierContent
Function	<i>global</i>
Harness	<i>global</i>
HierBox	<i>global</i> , HierContent
HierComponent	<i>global</i> , Define , HierContent
HierContent	HierBox , HierComponent
Inliner	<i>global</i> , Define , HierContent

Keyword	Used within the scope of
Join	Component , LComponent , Inliner , Splice , Eyelet , SVGComponent , HierComponent , HierContent , Config
LComponent	<i>global</i> , Define , HierContent
Module	<i>global</i>
Multicore	<i>global</i> , HierContent
Objects	Always , Config
Parameter	Define
Partner	Inliner , Config
Signal	<i>global</i>
Splice	<i>global</i> , Define , HierContent
SVGComponent	<i>global</i> , Define , HierContent
Wire	<i>global</i> , HierContent

A.2. Properties and Fixed Property Values

Property	Allowed values	Applies to	Repeat
Color	"syntax"	Component , Connector , Eyelet , HierComponent , Inliner , LComponent , Splice , SVGComponent , Wire	no
Ecfile	"syntax"	Cavity	no
Expr	"syntax"	Config	no
Href	"syntax"	Component , LComponent , SVGComponent , HierComponent , Inliner , Splice , Eyelet , Wire , Multicore , Connector , Cavity , Function , Harness , Signal , DBus , Module	yes
Image	"syntax"	Component , LComponent , SVGComponent , HierComponent , Inliner , Splice , Eyelet , Wire , Multicore , Connector , Cavity , Function , Harness , Signal , DBus , Module	yes

Property	Allowed values	Applies to	Repeat
Imagedsp	"syntax"	Component , LComponent , HierComponent	no
Index	"syntax"	Attributes	no
Name	"string"	Component , LComponent , SVGComponent , HierComponent , Inliner , Splice , Eyelet , Wire , Multicore , Connector , Cavity , Function , Harness , Signal , DBus , Module	no
Option	autocomplete	Function , Harness , Signal , DBus , Module	no
Parent	<i>ID</i>	Multicore	no
Shield	<i>ID</i>	Multicore	no
Style	"string"	Component , LComponent , SVGComponent , HierComponent , Inliner , Splice , Eyelet , Wire , Connector	no
Subtype	"string"	Component , LComponent , SVGComponent , HierComponent , Inliner , Splice , Eyelet , Wire , Multicore , Connector , Cavity , Function , Harness , Signal , DBus , Module	no
Symdef	"syntax"	SVGComponent , Cavity (of SVGComponents)	no
Type	male , female , invisible , half , anti	Connector	no
	in , out , halfdot , spliced	Cavity	no
	power , ground , logical , bus , hv	Wire	no
	twisted , shielded , twshielded	Multicore	no

Property	Allowed values	Applies to	Repeat
Video	"syntax"	Component , LComponent , SVGComponent , HierComponent , Inliner , Splice , Eyelet , Wire , Multicore , Connector , Cavity , Function , Harness , Signal , DBus , Module	yes

Last updated 2024-12-06 00:02:24 +0100